Importing important libraries for my analysis

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Importing Dataset for Analysis using pandas

```
df = pd.read_csv(r"C:\Users\DELL\Downloads\85334965-5736-457a-b8d4-a077e6872f84.csv")
```

```
df.head()
```

	TV	Radio	Social_Media	Sales
0	16.0	6.566231	2.907983	54.732757
1	13.0	9.237765	2.409567	46.677897
2	41.0	15.886446	2.913410	150.177829
3	83.0	30.020028	6.922304	298.246340
4	15.0	8.437408	1.405998	56.594181

```
df.describe()
```

	TV	Radio	Social_Media	Sales
count	4562.000000	4568.000000	4566.000000	4566.000000
mean	54.066857	18.160356	3.323956	192.466602
std	26.125054	9.676958	2.212670	93.133092
min	10.000000	0.000684	0.000031	31.199409
25%	32.000000	10.525957	1.527849	112.322882
50%	53.000000	17.859513	3.055565	189.231172
75%	77.000000	25.649730	4.807558	272.507922
max	100.000000	48.871161	13.981662	364.079751

Checking for Missing Values

```
df.isnull().sum()
```

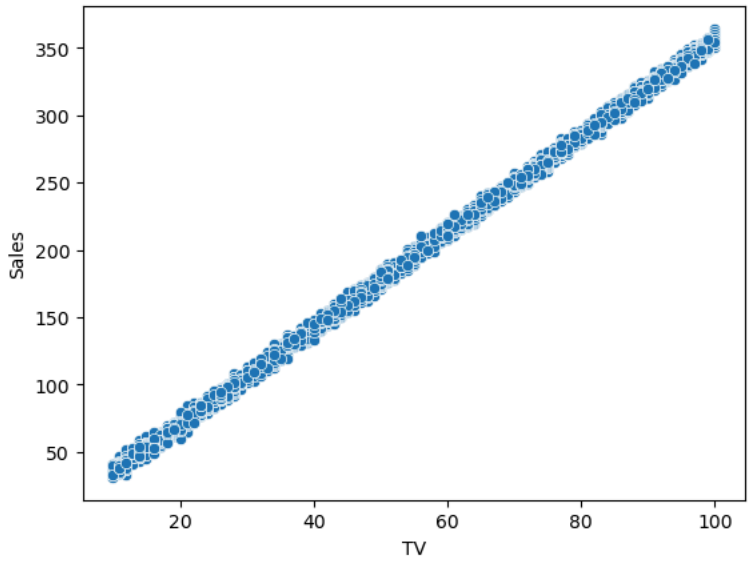
```
TV              10
Radio            4
Social_Media     6
Sales            6
dtype: int64
```

I will drop the missing values since they are less than I% of the entire dataset.

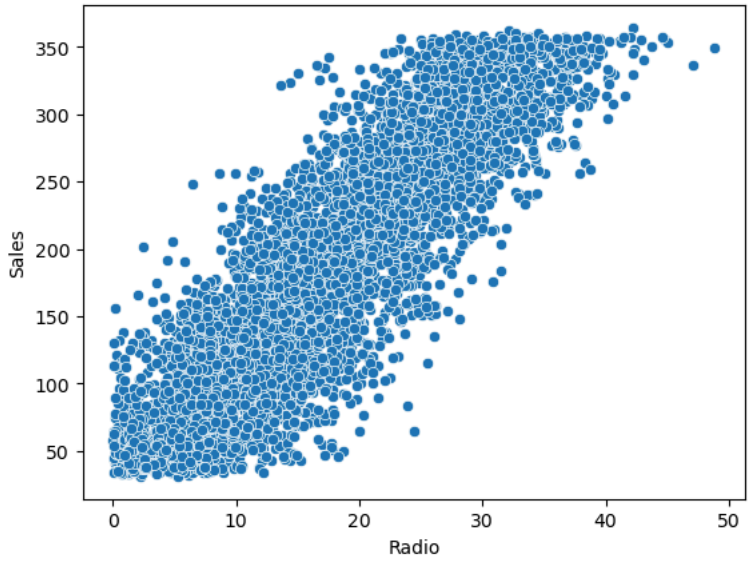
```
df = df.dropna()
```

Using scatterplot to identify which of the independent variables (TV,Radio,Social media) that is most correlated to the dependent varaables Sales

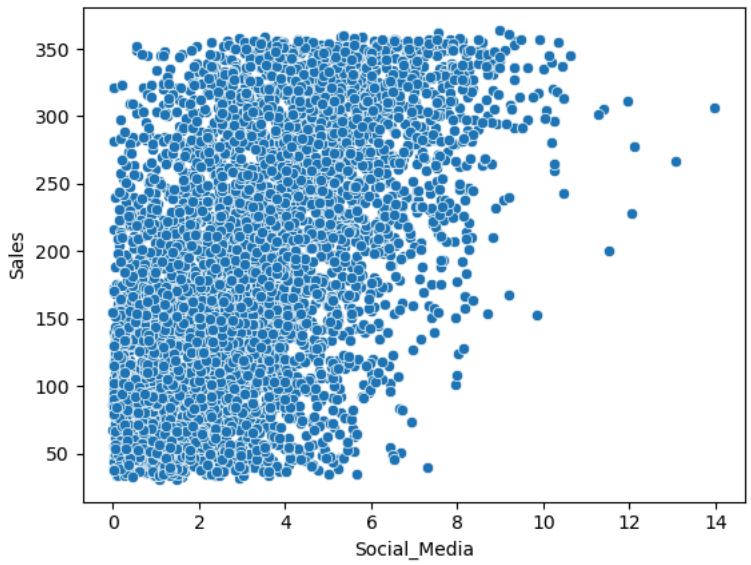
```
sns.scatterplot(x='TV', y='Sales', data=df)
plt.show()
```




```
sns.scatterplot(x='Radio', y='Sales', data=df)
plt.show()
```




```
sns.scatterplot(x='Social_Media', y='Sales', data=df)
plt.show()
```



From the above visualizations, TV is the most correlated variable with Sales

Building OLS regression model using statsmodels.

Features(X) = TV, Radio, Social media

Target(Y) = Sales

```
import statsmodels.api as sm
```

```
x = df[['TV', 'Radio', 'Social_Media']]
y = df['Sales']
```

```
X = sm.add_constant(x)
```

```
model = sm.OLS(y, X).fit()
```

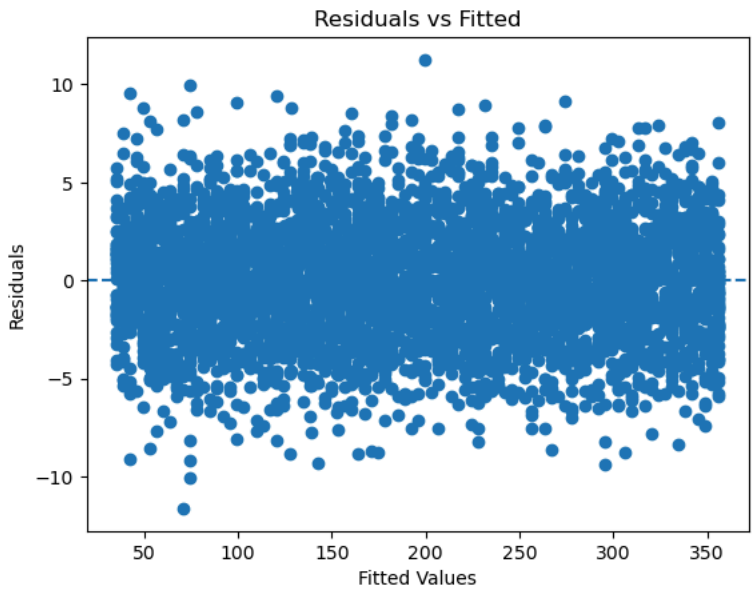
```
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.999
Model:                            OLS   Adj. R-squared:                  0.999
Method:                 Least Squares   F-statistic:                 1.505e+06
Date:                Tue, 09 Jun 2026   Prob (F-statistic):               0.00
Time:                        04:13:13   Log-Likelihood:                -11366.
No. Observations:                4546   AIC:                         2.274e+04
Df Residuals:                    4542   BIC:                         2.277e+04
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          -0.1340      0.103     -1.303      0.193      -0.336       0.068
TV              3.5626      0.003   1051.118      0.000       3.556       3.569
Radio          -0.0040      0.010     -0.406      0.685      -0.023       0.015
Social_Media    0.0050      0.025      0.199      0.842      -0.044       0.054
==============================================================================
Omnibus:                        0.056   Durbin-Watson:                   1.998
Prob(Omnibus):                  0.972   Jarque-Bera (JB):                0.034
Skew:                          -0.001   Prob(JB):                        0.983
Kurtosis:                       3.013   Cond. No.                         149.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Creating diagnostic plots to text linearity, Normality and Homoscedasticity

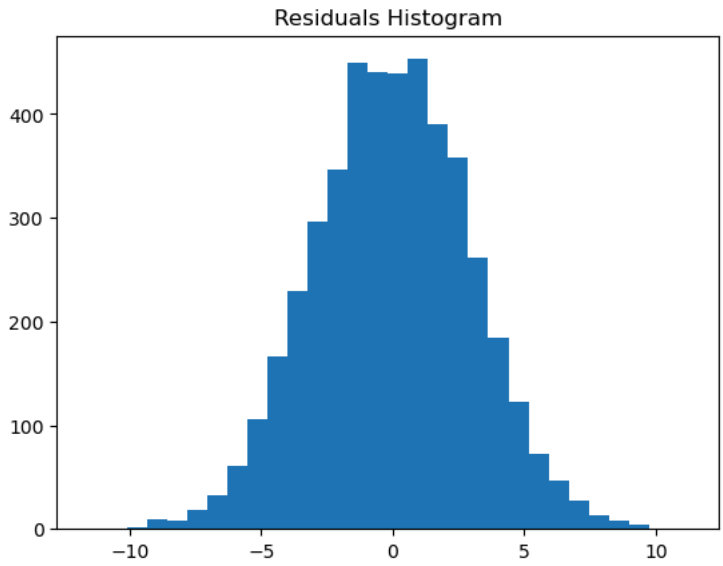
```
residuals = model.resid
fitted = model.fittedvalues
```

```
## checking for the Linearity

plt.scatter(fitted, residuals)
plt.axhline(y=0, linestyle='--')
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
plt.title('Residuals vs Fitted')
plt.show()
```



The relationship is linear because the residuals are randomly clustered around zero and there is no obvious pattern

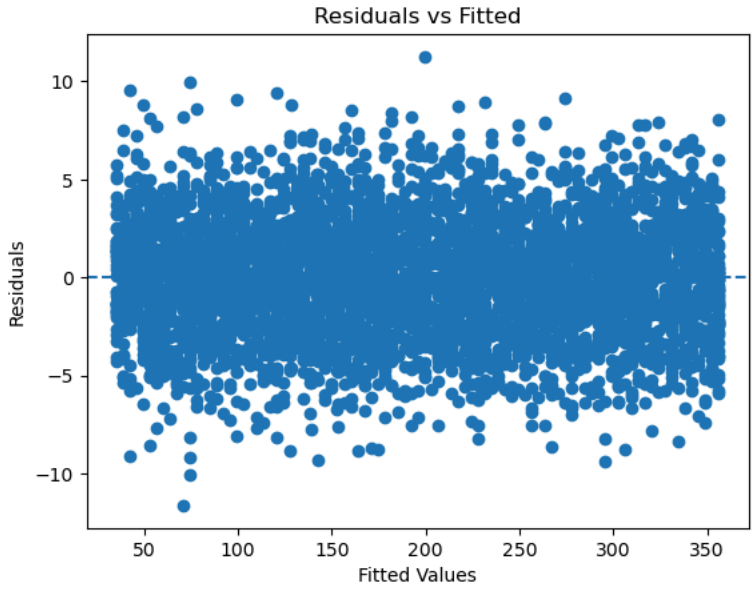
```
## checking for the Normality of the distribution

plt.hist(residuals, bins=30)
plt.title("Residuals Histogram")
plt.show()
```



The bell-shaped histogram shows that the distribution is Normal

```
## Checking for the Homoscedasticity of the dataset.

plt.scatter(fitted, residuals)
```

```
plt.axhline(y=0, linestyle='--')
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
plt.title('Residuals vs Fitted')
plt.show()
```



The roughly spread of the residuals across the fitted values shows the homoscedasticity of the dataset.

```
Start coding or generate with AI.
```

Interpreting R-Squared, coefficients and P_values in business context

From the model, R-squared is 0.999 which explains that there was 99.9% sales changes over time. Indicating that advertising channels (TV,Radio,social media) are strong drivers of sales.

Coefficient of: TV = 3.5626, Radio = -0.0040, Social-Media = 0.0050. This implies that for every 1 unit increase in TV advertising, Sales increases by 3.5626 units assuming Radio and social media spending remains constant. TV advertising has the greatest impact on Sales.

p_value for TV = 0.000 < 0.05, p_value for Radio = 0.685 > 0.05, p_value of Social media = 0.842 > 0.05. This implies that TV advertising channel really has great impact on sales while Radio and Social media advertising channels have little or no impact on sales.

```
Start coding or generate with AI.
```

Formulating a clear ROI - based recommendation for marketing budget allocation

```
tv_coef = model.params['TV']
tv_sales_contribution = (df['TV'] * tv_coef).sum()
tv_cost = df['TV'].sum()
tv_roi = ((tv_sales_contribution - tv_cost) / tv_cost) * 100
print(tv_roi)
```

```
256.25696271199195
```

```
Radio_coef = model.params['Radio']
Radio_sales_contribution = (df['Radio'] * Radio_coef).sum()
Radio_cost = df['Radio'].sum()
Radio_roi = ((Radio_sales_contribution - Radio_cost) / Radio_cost) * 100
print(Radio_roi)
```

```
-100.39703866458876
```

```
Social_Media_coef = model.params['Social_Media']
Social_Media_sales_contribution = (df['TV'] * Social_Media_coef).sum()
Social_Media_cost = df['Social_Media'].sum()
Social_Media_roi = ((Social_Media_sales_contribution - Social_Media_cost) / Social_Media_cost) * 100
print(Social_Media_roi)
```

```
-91.93269362570024
```